

```

using CasaToro.Consulta.Certificados.BL.Services;
using CasaToro.Consulta.Certificados.Entities;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;

namespace CasaToro.Consulta.Certificados.Web.Controllers
{
    [Authorize(Roles = "Admin")]
    public class AdminController : Controller
    {
        private readonly CertificateServiceExcel
_certificateServiceExcel;
        private readonly ProviderService _providerService;
        private readonly IWebHostEnvironment _webHostEnvironment;

        // Constructor del controlador que recibe instancias de los
servicios necesarios
        public AdminController(CertificateServiceExcel
certificateServiceExcel, ProviderService providerService,
IWebHostEnvironment webHostEnvironment)
        {
            _certificateServiceExcel = certificateServiceExcel;
            _providerService = providerService;
            _webHostEnvironment = webHostEnvironment;
        }

        // Acción que muestra la vista principal del administrador
public ActionResult UpdateInfo()
        {
            return View();
        }

        // Acción que muestra la vista para actualizar la información
básica
public ActionResult FormUnicProv()
        {
            return View();
        }

        // Acción que maneja la actualización de la información de los
certificados desde un archivo Excel
        [HttpPost]
        public IActionResult UpdateCertificateInfo(IFormFile file, string
infoType)
        {
            // Verificar si el archivo no es nulo y tiene contenido
            if (file != null && file.Length > 0)
            {
                // Procesar el archivo según el tipo de información
especificado
                if (infoType == "IVA")

```

```

        {
            var result =
            _certificateServiceExcel.AddInfoIvaFromExcel(file);
            if (!result.success) return Json(new { error =
result.message });
            return Json(new { result.message });
        }

        if (infoType == "ICA")
        {
            var result =
            _certificateServiceExcel.AddInfoIcaFromExcel(file);
            if (!result.success) return Json(new { error =
result.message });
            return Json(new { result.message });
        }

        if (infoType == "RTF")
        {
            var result =
            _certificateServiceExcel.AddInfoRtfFromExcel(file);
            if (!result.success) return Json(new { error =
result.message });
            return Json(new { result.message });
        }

        // Si el tipo de información no es soportado, devolver un
mensaje de error
        return Json(new { error = "Tipo de información no
soportado" });
    }

    // Si el archivo es nulo o está vacío, devolver un mensaje de
error
    return Json(new { error = "Ocurrio un error al procesar el
archivo" });
}

// Acción que muestra la vista para la administración de
proveedores
public ActionResult ProviderList()
{
    try
    {
        return View();
    }
    catch (Exception ex)
    {
        ViewBag.ErrorMessage = ex.Message;
        return View("Error");
    }
}

//accion para obtener la lista paginada de proveedores

```

```

        [HttpGet]
        public IActionResult GetProviders(int pageNumber = 1, int
datos    pageSize = 100, string? search = null)
        {
            try
            {
                // Obtener lista paginada con búsqueda en la base de
                var providers = _providerService.getProviders(pageNumber,
datos    pageSize, search);

                // Contar total de proveedores con filtro
                var totalProviders =
                _providerService.getProvidersCount(search);

                return Json(new
                {
                    providers,
                    currentPage = pageNumber,
                    pageSize,
                    totalProviders
                });
            }
            catch (Exception ex)
            {
                return Json(new { error = ex.Message });
            }
        }

        [HttpPost]
        public IActionResult UpdateProvider([FromBody] Proveedores_Master
provider)
        {
            try
            {
                // Verificar si el proveedor existe
                var existingProvider =
                _providerService.getProviderByNit(provider.Nit);
                if (existingProvider == null) return Json(new { error =
                "Proveedor no encontrado" });

                // Actualizar información del proveedor existente
                existingProvider.Nombre = provider.Nombre.ToUpper();
                existingProvider.Direccion = provider.Direccion;
                existingProvider.Correo = provider.Correo;
                existingProvider.Telefono = provider.Telefono;

                // Actualizar información del proveedor en la base de
                _providerService.UpdateProvider(existingProvider);
                return Json(new { message = "Información actualizada
datos    correctamente" });
            }
            catch (Exception ex)

```

```

        {
            return Json(new { error = ex.Message });
        }
    }

    //Accion para actualizar persona natural
    [HttpPost]
    [Route("/Admin/UpdateProviderNatural")]
    public IActionResult UpdateProviderNatural([FromBody]
Proveedores_Natural providerData)
    {
        try
        {
            if (providerData == null)
            {
                Console.WriteLine("UpdateProviderNatural: Provider
data is null");
                return Json(new { error = "Datos del proveedor no
recibidos." });
            }
            Console.WriteLine("UpdateProviderNatural payload: " +
Newtonsoft.Json.JsonConvert.SerializeObject(providerData));
            _providerService.UpdateNaturalInfo(providerData);
            return Json(new { message = "Información de persona
natural actualizada correctamente." });
        }
        catch (Exception ex)
        {
            return Json(new { error = "Error al actualizar la persona
natural: " + ex.Message });
        }
    }

    //accion para actualizar persona juridica
    [HttpPost]
    [Route("/Admin/UpdateProviderJuridica")]
    public IActionResult UpdateProviderJuridica([FromBody]
Proveedores_Juridica providerData)
    {
        try
        {
            if (providerData == null)
            {
                return Json(new { error = "Datos del proveedor no
recibidos." });
            }

            var pmaster =
_providerService.getProviderByNit(providerData.Nit);
            if (pmaster == null)
            {
                return Json(new { error = "El proveedor no esta
registrado en el sistema." });
            }

            _providerService.UpdateJuridicaInfo(providerData);
            return Json(new { message = "Información de persona
jurídica actualizada correctamente." });
        }
    }

```

```

        }
        catch (Exception ex)
        {
            return Json(new { error = "Error al actualizar la persona
jurídica: " + ex.Message });
        }
    }

    //accion para actualizar form general
    [HttpPost]
    [Route("/Admin/UpdateProviderFUCP")]
    public IActionResult UpdateProviderFUCP([FromBody]
Proveedores_FUCP providerData)
    {
        try
        {
            if (providerData == null)
                return Json(new { status = "error", message = "Datos
no recibidos." });

            var existing =
_providerService.getFUCPByNit(providerData.Nit);
            if (existing == null)
                return Json(new { status = "error", message = "El
proveedor no está registrado en el sistema." });

            _providerService.UpdateFUCPInfo(providerData);

            return Json(new { status = "success", message =
"Información FUCP actualizada correctamente." });
        }
        catch (Exception ex)
        {
            return Json(new { status = "error", message = "Error al
actualizar la informacion FUCP: " + ex.Message });
        }
    }

    //accion para consultar proveedor
    [HttpGet]
    public async Task<IActionResult> CheckProvider(string idNum,
string personType)
    {
        try
        {
            if (string.IsNullOrEmpty(idNum) ||
string.IsNullOrEmpty(personType))
                return Json(new { status = "error", message = "ID y
Tipo de persona son requeridos." });

            //verifica si el proveedor existe en la tabla
Proveedores_Master
            var providerMaster =
_providerService.getProviderByNit(idNum);

```

```

        //si no lo encuentra en proveedores_Master
        if (providerMaster == null)
        {
            return Json(new { status = "notFound", idNum });
        }

        //intenta obtener los detalles de persona (natural o
juridica) + FUCP
        dynamic naturalData = await
_providerService.getProviderDetails(idNum, "natural");
        dynamic juridicaData = await
_providerService.getProviderDetails(idNum, "juridica");

        bool natur = naturalData?.existNatu ?? false;
        bool juri = juridicaData?.existJuri ?? false;
        bool fucpN = naturalData?.existFUCP ?? false;
        bool fucpJ = juridicaData?.existFUCP ?? false;

        // Si existe registro como natural y se está consultando
juridica
        if (natur && personType == "juridica")
            return Json(new { status = "misMatch", registeredType
= "natural" });

        // Si existe registro como juridica y se está consultando
natural
        if (juri && personType == "natural")
            return Json(new { status = "misMatch", registeredType
= "juridica" });

        //si se encuentra en la tabla correspondiente al tipo de
persona
        if (personType == "natural" && natur)
            return Json(new { status = "foundDetail", data =
naturalData });

        if (personType == "juridica" && juri)
            return Json(new { status = "foundDetail", data =
juridicaData });

        //si se encuentra en proveedores_Master pero no en las
tablas de tipo de persona
        return Json(new { status = "foundMasterOnly", data =
providerMaster });
    }
    catch (Exception ex)
    {
        return Json(new { status = "error", message = "Error al
consultar proveedor: " + ex.Message });
    }
}

//accion para agregar proveedor natural

```

```

[HttpPost]
[Route("/Admin/AddProviderNatural")]
public async Task<IActionResult> AddProviderNatural([FromBody]
Proveedores_Natural provider)
{
    try
    {
        if (provider == null) return Json(new { error = "Datos
del proveedor no recibidos." });

        //validar la existencia del proveedor en la tabla
Proveedores_Master
        var pmaster =
_providerService.getProviderByNit(provider.Nit);
        if (pmaster == null) return Json(new { error = "El
proveedor no esta registrado en el sistema." });

        //validar que no exista ya el registro en
proveedor_natural
        dynamic existingNatural = await
_providerService.getProviderDetails(provider.Nit, "natural");
        if (existingNatural.existNatu) return Json(new { error =
"El proveedor ya tiene registrada su información como persona natural."
});

        //insertar registro de persona natural
_providerService.AddProveedorNatural(provider);
        return Json(new { message = "Proveedor de persona natural
registrado correctamente." });
    }
    catch (Exception ex)
    {
        return Json(new { error = ex.Message });
    }
}

//accion para agregar proveedor juridico
[HttpPost]
[Route("/Admin/AddProviderJuridica")]
public async Task<IActionResult> AddProviderJuridica([FromBody]
Proveedores_Juridica provider)
{
    try
    {
        if (provider == null) return Json(new { error = "Datos
del proveedor no recibidos." });

        //validar la existencia del proveedor en la tabla
Proveedores_Master
        var pmaster =
_providerService.getProviderByNit(provider.Nit);
        if (pmaster == null) return Json(new { error = "El
proveedor no esta registrado en el sistema." });

```

```

        //validar que no exista ya el registro en
proveedor_juridica
        dynamic existingJuridica = await
_providerService.getProviderDetails(provider.Nit, "juridica");
        if (existingJuridica.existJuri) return Json(new { error =
"El proveedor ya tiene registrada su información como persona jurídica."
});

        //insertar registro de persona juridica
_providerService.AddProveedorJuridica(provider);
return Json(new { message = "Proveedor de persona
jurídica registrado correctamente." });
    }
    catch (Exception ex)
    {
        return Json(new { error = ex.Message });
    }
}

//accion para agregar form general
[HttpPost]
[Route("/Admin/AddProviderFUCP")]
public IActionResult AddProviderFUCP([FromBody] Proveedores_FUCP
provider)
{
    try
    {
        if (provider == null) return Json(new { status = "error",
message = "Datos generales del proveedor no recibidos." });

        //validar la existencia del proveedor en la tabla
Proveedores_Master
        var pmaster =
_providerService.getProviderByNit(provider.Nit);
        if (pmaster == null) return Json(new { status = "error",
message = "El proveedor no esta registrado en el sistema." });

        //validar que no exista ya el registro en
proveedores_FUCP
        var existingFUCP =
_providerService.getFUCPByNit(provider.Nit);
        if (existingFUCP != null) return Json(new { status =
"error", message = "El proveedor ya tiene informacion registrada en
FUCP." });

        //insertar registro en FUCP
_providerService.AddProveedorFUCP(provider);
return Json(new { status = "success", message =
"Proveedor registrado en FUCP correctamente." });
    }
    catch (Exception ex)
    {
        return Json(new { status = "error", message = "Error al
agregar FUCP: " + ex.Message });
    }
}

```



```

    }
}

[HttpPost]
[Route("/Admin/UploadDocuments")]
public async Task<IActionResult> UploadDocuments(string Nit,
string personType, bool isOEA)
{
    try
    {
        var files = Request.Form.Files;
        if (files.Count == 0) return Json(new { status = "error",
message = "No se recibieron archivos." });

        foreach (var file in files)
        {
            string categoria = file.Name;

            string rutaRel = await
_providerService.SaveDocuments(file, Nit, personType, categoria,
_webHostEnvironment.WebRootPath);

            _providerService.SaveDocumentMD(Nit, categoria,
file.FileName, rutaRel, isOEA);
        }

        return Json(new { status = "success", message = "Archivos
cargados y registrados correctamente." });
    }
    catch (Exception ex)
    {
        return Json(new { status = "error", message = ex.Message
});
    }
}

//accion para restaurar contrasena de proveedor
[HttpGet]
public IActionResult RestoreProviderPassword(string nit)
{
    try
    {
        if (string.IsNullOrEmpty(nit)) return Json(new { error =
"NIT del proveedor no recibido" });

        // Verificar si el proveedor existe
        var existingProvider =
_providerService.getProviderByNit(nit);
        if (existingProvider == null) return Json(new { error =
"Proveedor no encontrado" });
        // Restaurar contraseña del proveedor
        _providerService.RestoreProviderPassword(nit);
    }
}

```

```
        return Json(new { message = "Contraseña restaurada
correctamente" });
    }
    catch (Exception ex)
    {
        return Json(new { error = ex.Message });
    }
}
}
```